Loops

Repeating code with while, do-while and for

Contents

Concepts	Why loops?, The three loops in C, How a for loop runs, Controlling loops
Example 1	Print the numbers from 1 to 5
Example 2	Sum of the numbers from 1 to 10
Example 3	Factorial
Example 4	Repeat until the input is valid
Example 5	while vs do-while
Example 6	for loops with several variables
Example 7	Printing numbers separated by commas
Example 8	Star triangle (nested loops)
Example 9	Stopping before integer overflow (flag + break)

Why loops?

A loop runs the same block of code several times. The repeated work does not have to be identical each time: usually something small changes on every pass, such as a counter or the number being printed. **Rule of thumb: if something repeats, use a loop.** One pass through the loop body is called an **iteration**.

Loops and branching (`if`, `switch`) are the two ways to control the flow of a program. Branching chooses *which* path to take, while loops choose *how many times* to run a path.

The three loops in C

Loop	Condition checked	Runs at least once?
`while`	before each iteration	No
`do-while`	after each iteration	Yes
`for`	before each iteration	No

```
while (condition) {
    // repeats while condition is true
}

do {
    // runs once, then repeats while condition is true
} while (condition);      // note the semicolon

for (initialization; condition; update) {
    // repeats while condition is true
}
```

How a for loop runs

- **Initialization** runs only once, before the first iteration.
- **Condition** is checked before *every* iteration. If it is false at the start, the body never runs.
- **Update** runs after each iteration, before the condition is checked again.
- Any part may contain several expressions separated by commas, for example `for (i = 0, j = 1; ...; i++, j++)`.
- In the update part, `i++` and `++i` give the same result, because the value is not used in a comparison there. Inside a condition such as `while (count++ < 10)` the position does matter.

Controlling loops

- `break;` leaves the loop immediately.
- `while (1)` creates an infinite loop, because 1 always means true in C. It must be ended with `break`.
- Counters and sums should be given a starting value (usually 0 or 1) before the loop starts. Otherwise they contain garbage values.

EXAMPLE 1

Print the numbers from 1 to 5

Use a while loop to print every integer from 1 to 5.

CODE

```c
#include <stdio.h>

int main() {
    int i = 1;

    while (i <= 5) {
        printf("%d\n", i);
        i++;                /* without this, the loop never ends */
    }
    return 0;
}
```

SAMPLE RUN

```
1
2
3
4
5
```

EXAMPLE 2

Sum of the numbers from 1 to 10

Add up all integers from 1 to 10 using a while loop.

CODE

```c
#include <stdio.h>

int main() {
    int sum = 0;
    int i = 1;

    while (i <= 10) {
        sum += i;
        i++;
    }
    printf("The sum of numbers from 1 to 10 is: %d\n", sum);
    return 0;
}
```

SAMPLE RUN

```
The sum of numbers from 1 to 10 is: 55
```

EXAMPLE 3

Factorial

Read a non-negative integer and calculate its factorial (n! = n x (n-1) x ... x 1).

CODE

```c
#include <stdio.h>

int main() {
    int num, factorial = 1;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
        return 1;
    }

    while (num > 1) {
        factorial *= num;
        num--;
    }

    printf("The factorial is: %d\n", factorial);
    return 0;
}
```

SAMPLE RUN

```
Enter a positive integer: 6
The factorial is: 720
```

Note. An `int` overflows after 12!, so larger inputs give wrong results. Use `long long` for bigger values.

EXAMPLE 4

Repeat until the input is valid

Keep asking for a number until the user enters a positive one.

CODE

```c
#include <stdio.h>

int main() {
    int number;

    printf("Enter a positive number: ");
    while (1) {                 /* 1 is always true: infinite loop */
        scanf("%d", &number);
        if (number > 0) {
            break;              /* valid input: leave the loop */
        }
        printf("Invalid input. Please enter a positive number: ");
    }
    printf("You entered a valid number: %d\n", number);
    return 0;
}
```

SAMPLE RUN

```
Enter a positive number: -4
Invalid input. Please enter a positive number: 0
Invalid input. Please enter a positive number: 9
You entered a valid number: 9
```

EXAMPLE 5

while vs do-while

Print "Hello!" three times with each loop type. In the do-while version, the body runs once before the condition is checked for the first time.

CODE

```c
#include <stdio.h>

int main() {
    int count;

    /* while: the condition is checked first */
    count = 0;
    while (count++ < 3) {
        printf("while: Hello!\n");
    }

    /* do-while: the body runs first, then the condition is checked */
    count = 0;
    do {
        printf("do-while: Hello!\n");
    } while (++count < 3);

    /* do-while runs once even when the condition is false from the start */
    count = 100;
    do {
        printf("do-while ran once although count = %d\n", count);
    } while (count < 3);

    return 0;
}
```

SAMPLE RUN

```
while: Hello!
while: Hello!
while: Hello!
do-while: Hello!
do-while: Hello!
do-while: Hello!
do-while ran once although count = 100
```

Note. In the while loop the counter is compared first and increased afterwards (`count++`). In the do-while loop the first print has already happened, so the counter is increased *before* the comparison (`++count`). Both versions print three lines.

EXAMPLE 6

for loops with several variables

A for loop can initialize and update more than one variable. The condition can combine tests with `||` or `&&`.

CODE

```c
#include <stdio.h>

int main() {
    int i, j;

    /* runs while AT LEAST ONE of the two conditions is true */
    for (i = 0, j = 1; i < 10 || i < j; i += 2, j++) {
        printf("i = %d, j = %d\n", i, j);
    }
    return 0;
}
```

SAMPLE RUN

```
i = 0, j = 1
i = 2, j = 2
i = 4, j = 3
i = 6, j = 4
i = 8, j = 5
```

EXAMPLE 7

Printing numbers separated by commas

Print the numbers from 0 to 9 separated by commas, without a comma after the last number. An if-else inside the loop handles the last iteration differently.

CODE

```c
#include <stdio.h>

int main() {
    int count = 0;

    while (count < 10) {
        if (count == 9) {
            printf("%d", count);        /* last number: no comma */
        } else {
            printf("%d, ", count);
        }
        count++;
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

EXAMPLE 8

Star triangle (nested loops)

Print a triangle of stars where row 1 has one star, row 2 has two stars, and so on. The outer loop counts the rows and the inner loop prints the stars in each row.

CODE

```c
#include <stdio.h>

int main() {
    int rows = 5;
    int i, j;

    for (i = 1; i <= rows; i++) {           /* one iteration per row   */
        for (j = 0; j < i; j++) {           /* i stars in row i        */
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

SAMPLE RUN

```
*
**
***
****
*****
```

Note. The original class notes wrote a separate for loop for each row. The nested loop above does the same job for any number of rows.

EXAMPLE 9

Stopping before integer overflow (flag + break)

Add the numbers 1 to 1000, but stop and report an error if the sum is about to exceed the safe limit for an `int`. A flag variable remembers whether the loop was stopped early.

CODE

```c
#include <stdio.h>

int main() {
    int overflowFlag = 0;
    int i;
    int total = 0;

    for (i = 1; i <= 1000; i++) {
        if (total < 2000000000) {
            total += i;
        } else {
            overflowFlag = 1;
            break;
        }
    }

    if (!overflowFlag) {
        printf("Total = %d\n", total);
    } else {
        printf("Integer overflow error!\n");
    }
    return 0;
}
```

SAMPLE RUN

```
Total = 500500
```